OBFP: Optimized Blockchain-Based Fair Payment for Outsourcing Computations in Cloud Computing

Chao Lin, Debiao He, *Member, IEEE*, Xinyi Huang, and Kim-Kwang Raymond Choo, *Senior Member, IEEE*

***Abstract*— Outsourcing computations have been widely used to meet the growing computing demands, although achieving trust in an untrusted (or a zero-trust) environment can be challenging in practice. Fair payment, a candidate solution, can potentially facilitate fair trading among outsourcing computation participants such as users and workers. However, most existing solutions including traditional e-cash-based or blockchain-based, may potentially compromise the worker's fairness (i.e., does not achieve robust fairness, since trusted third parties are required during the trading process), or involve heavy zero-knowledge proofs (ZKPs, with significant computation costs). To mitigate these limitations, we propose a system model of an optimized blockchain-based fair payment (OBFP) for outsourcing computations. Then, we construct a ZKP-free solution based on blockchain by combining any secure commitment, accumulator, and symmetric encryption schemes, as well as a hash function. To demonstrate the utility of our proposed OBFP system, we provide security analysis, performance evaluation and a comparison with existing popular solutions. Specifically, the cryptographic tools are instantiated as commitment (Perdesen commitment), accumulator (RSA-based accumulator), and symmetric encryption (a concrete scheme with the indistinguishability under chosen-plaintext attack (IND-CPA) security), and a hash function (Keccak-256). The prototype is implemented in COSBench and Remix to analyze cloud scalability and concurrency, as well as gas cost.**

***Index Terms*— Blockchain, outsourcing computations, fair payment, smart contract, robust fairness.**

Manuscript received April 14, 2020; revised November 9, 2020 and March 15, 2021; accepted April 7, 2021. Date of publication April 16, 2021; date of current version June 2, 2021. The work was supported in part by the National Key Research and Development Program of China under Grant 2019QY0800; in part by the National Natural Science Foundation of China under Grant 61972294, Grant 61932016, Grant 62032005, Grant 61872089, and Grant 61972094; in part by the Science Foundation of Fujian Provincial Science and Technology Agency under Grant 2020J02016; in part by the Special Project on Science and Technology Program of Hubei Province under Grant 2020AEA013; in part by the Natural Science Foundation of Hubei Province under Grant 2020CFA052; and in part by the Wuhan Municipal Science and Technology Project under Grant 2020010601012187. The work of Kim-Kwang Raymond Choo was supported only by the Cloud Technology Endowed Professorship. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Lejla Batina. *(Corresponding author: Debiao He.)*

Chao Lin is with the Fujian Provincial Key Laboratory of Network Security and Cryptology/Center for Applied Mathematics of Fujian Province, College of Mathematics and Informatics, Fujian Normal University, Fuzhou 350117, China, also with the State Key Laboratory of Cryptology, Beijing 100878, China, also with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China (e-mail: linchao91@fjnu.edu.cn).

Debiao He is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, also with the Shandong Provincial Key Laboratory of Computer Networks, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China, and also with the Shanghai Key Laboratory of Privacy-Preserving Computation, MatrixElements Technologies, Shanghai 201204, China (e-mail: hedebiao@163.com).

Xinyi Huang is with the Fujian Provincial Key Laboratory of Network Security and Cryptology/Center for Applied Mathematics of Fujian Province, College of Mathematics and Informatics, Fujian Normal University, Fuzhou 350117, China, and also with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China (e-mail: xyhuang81@gmail.com).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security and the Department of Electrical and Computer Engineering, The University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: raymond.choo@fulbrightmail.org).

Digital Object Identifier 10.1109/TIFS.2021.3073818

I. Introduction

CONVENTIONAL single-server computing paradigm is incapable of meeting the demands of modern-day societal and computing needs, and consequently this has contributed to the development and adoption of distributed computing [1].

Distributed computation enables users (even resource-limited entities) to complete complex computation tasks with high efficiency, although outstanding trust concerns may impede its widespread deployment. For example, in the "lazy worker" concern, users cannot be assured that the worker will correctly execute the entire sub-task, since they are not incentivized to complete their entire workload (e.g. only execute partial sub-task) but obtain the full fees. The second concern is about the "dishonest user", where workers are not certain if they will be paid on schedule after the sub-tasks are completed. Thus, if the user and the worker distrust each other, payment becomes extremely challenging for outsourcing computation with fairness.

Fair payment, one of solutions to solving the above concerns, has been researched for decades. Early work on fair payment (e.g. [2]–[5]) only considered the lazy-but-honest workers, which is not a reasonable assumption. In 2002, Chen *et al.* [1] took lazy-and-partially-dishonest workers into account and proposed an efficient fair payment system. Unfortunately, Chen *et al.*'s proposal still relies on a bank (i.e. a trusted third party, TTP) during the payments. The TTP is always involved in the exchange and thus becomes a bottleneck.

To avoid relying on a TTP, Asokan *et al.* [6] defined the concept of optimistic fairness, where the TTP only intervenes

1556-6021 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

when the dispute happens. Along this line, there are many constructions [7]–[11] have been proposed, among which Huang et al.'s [10] construction first integrated blockchain into fair payment systems for outsourcing computations. This has highlighted the potential of blockchain in fair payment for outsourcing computations, but unfortunately their proposal is also unable to achieve the robust fairness (first defined in [12]). The robust fairness refers to that, during the process of trading, the user and worker can achieve fair exchange without any TTP.

There exist some blockchain-based fair payment systems having achieved the robust fairness, but they are faced with some barriers when being applied into outsourcing computations. For instance, the system designed in [13] realizes a robust fair exchange for digital goods and it is implemented for proofs of retrievability, but regrettably the performance of zero-knowledge proof (ZKP) in this system brings significant computation costs for the participants. Some ZKP-free solutions such as [12], [14], [15], without involving the intractable zero-knowledge proof, can efficiently lower the transaction fees in blockchain and computation costs. However, most of them are constructed based on the merkle tree and hence a worker needs to reply data of some sibling nodes to the user. This indeed damages the interest of the worker, and hence we argue that they are not completely consistent with the robust fairness.

Contributions: In this paper, we propose an optimized blockchain-based fair payment (OBFP) system model, and then integrate cryptographic techniques (including commitment, accumulator, symmetric encryption, and hash functions) with smart contract to construct a new OBFP system. The proposed system can satisfy both robust fairness and ZKP-free efficiency, and hence it is more utility than existing solutions. We also prove the security of our OBFP on basis of the binding, hiding properties of a commitment as well as collusion resistance of accumulator. Moreover, we further instantiate these cryptographic techniques as Perdesen commitment, RSA-based accumulator, a symmetric encryption scheme with the indistinguishability under chosen-plaintext attack (IND-CPA) security, and Keccak-256 hash function, and apply our OBFP into the outsourcing offline dictionary guess proposed by Wang *et al.* [16]. Our performance analysis results demonstrate the benefits of our proposal.

Outline of this paper: Section II and III describe the related work and involved preliminaries in our OBFP, respectively. Then in Section IV, we propose our OBFP with a high-level description, design of smart contract and details of system construction. To further demonstrate our OBFP, we give a concrete application and performance evaluation in Section V. Finally, we conclude this paper in Section VI.

II. Related Work

To address trust concerns of outsourcing computations, fair payment has been well studied in recent decades. In 2001, Golle and Mironov [5] first introduced the concept of ringers to deal with the "lazy worker" concern. While the ringers can help the user to confirm if the worker has indeed completed its whole computation task, they generally assume that the user is fully trusted by the worker. Such an assumption violates the basic requirement of outsourcing computation, that is, users and workers do not fully trust each other. On the "dishonest user" concern, Carbunar and Tripunitara have proposed several fair payment schemes such as [3], [4] to ensure the right of workers, namely, receiving deserved payments after completing the computation task. However, these schemes require an inefficient cut-and-choose protocol involving huge time costs, which is impractical for real deployments [1].

Another common insufficiency of above proposals (i.e. [3]–[5]) is assuming that workers are lazy-but-honest, which has been pointed out unreasonable by Chen *et al.* [1]. Thus, Chen *et al.* [1] considered the lazy-and-partially-dishonest workers and proposed a novel fair conditional payment system. Although their solution was more efficient than the prior systems, their adopted ringer still needs to compute the complex "inversion of one-way function" class. Besides, the bank-based e-cash may be a bottleneck during the application, for the bank is unwilling to implement it without being paid with high transaction fees.

Huang *et al.* [10] integrated the commitment-based sampling (CBS, introduced in [17]) into Bitcoin and proposed the first blockchain-based fair payment system for outsourcing computations. However, a TTP is unavoidable for dealing with the dispute, and hence its provided fairness is only optimistic but not robust. Even worse, the transaction delay (i.e. average 10 minutes for each block generation in Bitcoin) in their proposal will be intolerable in some scenarios with a high real-time requirement [18]–[20].

Fair payment for other scenarios (e.g. digital goods exchange, outsourcing service) can be classified into two types, that is, ZKP-based and ZKP-free. These protocols may be perhaps applied into outsourcing computations, but there exist some barriers explained as follows. Among the ZKP-based fair payment, the first solution was proposed by Bowe,[1] which is designed for efficiently selling solutions of Sudoku puzzles. However, this design is not secure for the common reference string is generated by the verifier [13]. Although Campanelli *et al.* [13] adopt subversion-NIZK to solve this problem and achieve the robust fairness (for the smart contract can be regarded as a TTP), the used ZKP brings significant on-chain complexity and also the huge computation costs of participants.

As the ZKP-free solutions, Dziembowski *et al.* [14] introduced the concept of "proofs of misbehavior" (PoM) to avoid the use of heavy ZKP. Their proposal (named as FairSwap) can reduce the on-chain complexity and suits the practical smart contract-based solutions, but it compromises the fairness from the perspective of worker for its verifiability is based on merkle tree. To reply the user's challenges such as data x_i, the worker needs to send data (e.g. x_{i+1}) of the sibling node in the merkle tree. That is, a malicious user can easily get all the data with only paying half the entire fee. Also

[1] https://github.com/zcash-hackworks/pay-to-sudoku

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

caused by the feature of merkle tree, data privacy will be compromised in the judge contract. This is due to the data of sibling node should be provided even it is correct. Subsequently, Eckey *et al.* [11] incorporated an interactive dispute resolution sub-protocol to extend the FairSwap and obtain a novel protocol (i.e. OptiSwap). Although OptiSwap improves the optimistic execution of the FairSwap (e.g. improving the computation overhead of two honest parties), it still uses the PoM tool and hence compromising the fairness of worker as discussed above.

In addition to the above issues, the merkle tree-based solutions [12], [15] are also faced with the following issues. The number of transactions is not a constant, which is affected by the user' challenges. Once the number of challenges is large, the delay will be unacceptable especially in Bitcoin (with about 10 minutes for each block generation). Moreover, the metadata are required to record into the blockchain, which will cause a huge storage burden for blockchain, and hence brings large expense such as gas costs in Ethereum. Recently, there are also other ZKP-free solutions proposed in [21], [22], namely, using Trusted Execution Environments. However, in this type of proposals, users may maliciously drop out without paying for the execution. Although executing a few cycles at a time can help mitigating this risk, it comes at the cost of both bandwidth and enclave execution time.

III. Preliminaries

This section presents the involved notations, cryptographic building blocks, and the system model in our OBFP system.

A. Notations

Let $\mathbb{G}$ denote a cyclic group of prime order p, and let $\mathbb{Z}_p$ denote the ring of integers modulo p. Let $\mathbb{Z}_p^*$ denote $\mathbb{Z} \setminus \{0\}$. Generators of $\mathbb{G}$ are denoted by $P, Q \in \mathbb{G}$. Let $\oplus$ denote the operation of Exclusive OR. We use bold font to denote sets, e.g., $\mathbf{S} \in \mathbb{F}^n$ is a set with elements $(S_1, \ldots, S_n)$, where $S_i \in \mathbb{F}$. We call a probabilistic algorithm running in polynomial time as a $\mathcal{PPT}$ algorithm. The notation $D_1 \overset{c}{\approx} D_2$ represents that two probability distributions D_1 and D_2 are computationally indistinguishable (i.e. resisting against a $\mathcal{PPT}$ distinguisher). In addition, we let λ denote a security parameter, pp a public parameter, (m, m_1, m_2) different messages, r a randomness, (cm, cm_1, cm_2) different commitments, X, Y, Z some finite sets, h_k, f some computable functions.

B. Cryptographic Building Blocks

1) Commitment: A commitment scheme [23] permits someone (e.g. Cindy) to commit to a chosen value while remaining hidden to others. Subsequently, Cindy can reveal the committed value later. There are three algorithms in the commitment scheme, namely, setup CSetup, commitment CCom and verification CVerf, with three properties of *completeness*, *binding* and *hiding*.

- *Completeness*. For all m and r,
$$Pr\left(\begin{array}{c} pp \leftarrow \mathsf{CSetup}(\lambda); \\ cm \leftarrow \mathsf{CCom}(pp, m, r) : \\ \mathsf{CVerf}(pp, cm, m, r) = 1 \end{array}\right) = 1.$$
- *Binding*. There is a negligible function $negl(\lambda)$ such that all non-uniform probabilistic polynomial time adversaries $\mathcal{PPT}$ $\mathcal{A}$,
$$Pr\left(\begin{array}{c} pp \leftarrow \mathsf{CSetup}(\lambda); \\ (cm, r, m_1, m_2) \leftarrow \mathcal{A}(pp) : \\ \mathsf{CVerf}(pp, cm, m_1, r) = 1 \wedge \\ \mathsf{CVerf}(pp, cm, m_2, r) = 1 \wedge \\ m_1 \neq m_2 \end{array}\right) \leq negl(\lambda).$$
- *Hiding*. For all pk generated by $\mathsf{CSetup}(\lambda)$, $\forall m_1, m_2$ of equal length, and $\forall r$, the following probability distributions are computationally indistinguishable:
$$\{cm_1 \leftarrow \mathsf{CCom}(pp, m_1, r) : cm_1\} \overset{c}{\approx}$$
$$\{cm_2 \leftarrow \mathsf{CCom}(pp, m_2, r) : cm_2\}.$$

2) One-Way Accumulators: One-way Accumulators [24], [25] allow accumulating multiple values into one single value with the ability to generate a witness for each value. This witness can prove that the corresponding value has been indeed accumulated. Generally, the accumulator scheme consists of four algorithms (i.e. setup ASetup, evaluation AEval, witness generation AWit, and verification AVerf) and it was defined as a family of one-way hash functions with an additional special property, namely, *quasi-commutativeness*.

- *One-way hash functions*. A family of one-way hash functions consists of infinite sets of functions $\{\mathcal{H}_\tau\}_{\tau \in \mathbb{N}}$, where $\mathcal{H}_\tau = \{h_k : X_k \times Y_k \rightarrow Z_k\}$, with the following properties.
  1) *Computability*. $\forall \tau$ and $\forall h_k \in \mathcal{H}_\tau$, $h_k(\cdot, \cdot)$ is computable in polynomial time of $\mathcal{O}(\lambda)$.
  2) *Collusion Resistance*. $\forall$ $\mathcal{PPT}$ algorithm $\mathcal{A}$, we have
$$Pr\left(\begin{array}{c} h_k \in \mathcal{H}_\tau; x \in X_k; y, y' \in Y_k; \\ x' \leftarrow \mathcal{A}(\lambda, \tau, x, y, y') : \\ h_k(x, y) = h_k(x', y') \end{array}\right) \leq negl(\lambda).$$
- *Quasi-commutativeness*. A function $f : X \times Y \rightarrow X$ satisfies this property if
$$\forall x \in X, \forall y_1, y_2 \in Y, f(f(x, y_1), y_2) = f(f(x, y_2), y_1).$$
- *One-way accumulators*. A family of one-way accumulators is a family of one-way hash functions that owns the quasi-commutativeness.

C. System Model

In the system model of an OBFP (see Fig. 1), there are four entities (i.e. "User", "Cloud", "Worker" and "Blockchain"), together with two communication methods (i.e. "off-chain" and "on-chain"). All of these entities are driven by interests, that is, they are not entirely trusted and may behave maliciously for their interests. The user (resp. worker) may be malicious and obtains the task results without payments (resp. receives service fee without completing the task). The

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

cloud will provide its computation or storage services for obtaining fees, but it may maliciously modify or delete some storing data to save costs. In the blockchain, the untrusted recording nodes will maintain a trusted blockchain ledger due to their own profits and the adopted consensus mechanism. The "off-chain" is a secure communication using a cryptographic protocol such as Transport Layer Security (TLS), and the "on-chain" is publishing a transaction using Peer-2-Peer (P2P) communication.

- User: A user in the OBFP system, may be some resource-constrained devices (e.g. mobile phone), could issue computation tasks into the smart contract (SC) chained in blockchain. Considering the computation task may involve large data, we adopt a cloud to store some large task data and only submitting the index path and hash value of the data into the SC. In some blockchain systems such as Ethereum, each step of triggering a smart contract requires fees, and hence it will cause high costs once storing and computing complex instructions or data results [26], [27]. Thus, our adopted storage method can efficiently reduce the fees caused by storing data in SC. This entity may be dishonest, meaning that he/she may postpone or reject the payment after receiving the task results.
- Cloud: As mentioned above, this type of participants is responsible for storing the original task data. The cloud will correctly respond to the user's and worker's requests (i.e. storage or retrieve) via the "off-chain" method, but it may be malicious to modify or delete the stored task data. In our OBFP, we store the index path and hash value of task data in the SC, where the former is used for conveniently retrieving task data from the cloud and the latter efficiently checking the integrity of task data.
- Worker: These participants are generally with sufficient computation capacity to perform computation tasks. A worker can get the index path and hash value of computation task from the smart contract, where the former is used to retrieve the large task data from the cloud and the latter is to check the integrity of retrieved data. The worker who executes a computation task correctly will get the service fee paid by the user and refund its guarantee deposit after a certain locking time. In contrast, a malicious worker who provides invalid task results will be punished, that is, their guarantee deposits will be transferred to the user as a compensation.
- Blockchain: This entity is maintained by a number of recording nodes, which are named as miners in public blockchain and permissioned nodes in permissioned one. In the OBFP system, any blockchain system supporting the functionality of smart contract (e.g. Ethereum and Hyperledger Fabric) can be adopted. A smart contract is deployed in the blockchain and then automatically executed once being triggered by the user or worker. The role of smart contract in OBFP is to link up the user and worker, that is, it can play the role of an interceder deciding where the paid service fee and guarantee deposit will be transferred to (e.g., user or worker). In addition, the communication method of both the user and worker contact with blockchain is "on-chain".

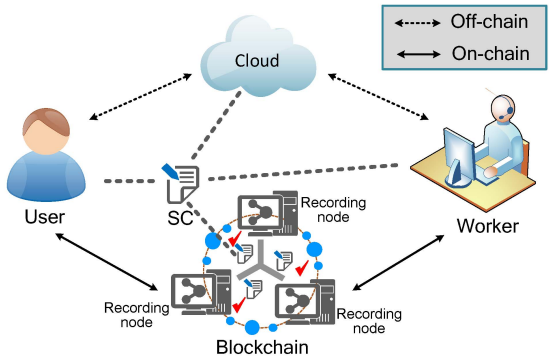


Fig. 1. System model of an optimized blockchain-based fair payment.

The outsourcing computation model can be formally defined as follows [1], [5]. Denote a computation task as $T = \langle f, \mathbf{S}, \mathbf{V} \rangle$, where the task T is to evaluate a function f on a finite domain $\mathbf{S}$, where $f : \mathbf{S} \rightarrow \mathbf{V}$ and $\mathbf{V}$ is a set of values interested by a user. A worker's goal is to compute $f(x), \forall x \in \mathbf{S}$ and returns those x values satisfying $f(x) \in \mathbf{V}$. This model could be directly applied into different computing scenarios by using the corresponding practical function f, for example, large prime searching uses f as a prime search function, or offline dictionary guessing uses f as a concrete hash function. Note that the $\mathbf{V}$ is initialized diversely in different scenarios, for example, it is set as N (i.e., the set of natural numbers) in large prime searching, but some targeted hash values in dictionary guessing. We also assume that $\mathbf{V}$ is some known items to the user obtained through other means without computing by itself.

On basic of the above system and outsourcing computation models, we focus on proposing an OBFP for outsourcing computations with the following goals.

1) Guarantee the robust fairness of both the user and worker without any TTP, including that they can get only what they deserve.
2) Avoid heavy ZKP, and reduce the communication and computation costs of the user and worker.
3) Lower transaction fees, ensuring that the transaction number in blockchain is constant even in different types of outsourcing computations.
4) Provide a more friendly experience for both the user and worker, namely, resisting to malicious behaviors including postpone payments and delay submitting results.

IV. Our Construction

According to the model definitions in Section III, we will present the construction of our OBFP. But before that, for clarity, we will first introduce our OBFP in a high-level description, and describe its involved smart contract design.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

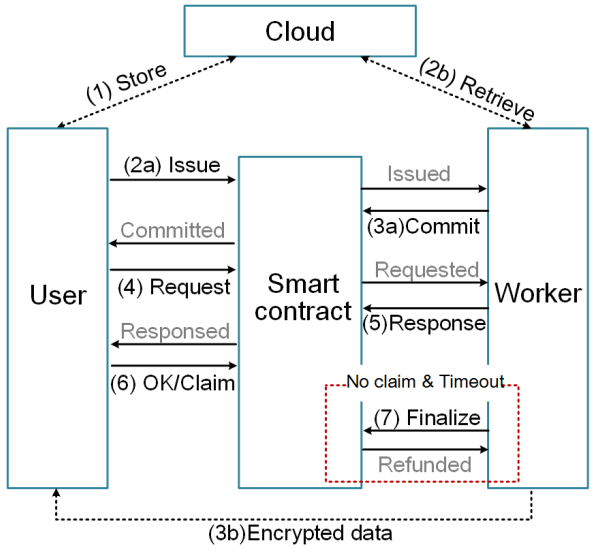


Fig. 2. Outline of our OBFP system.

Algorithm 1 OBFP(υ)

```
// Constructor, automatically triggered when deploying.
struct Task
{
    user; // address, the user's identity
    worker; // address, the worker's identity
    f_des; // string, description of the function
    h_s; // uint256, the hash value of S
    P_des; // string, description of the index path in cloud
    k; // uint256, the number of solutions
    h; // uint256, accumulator value of responses
    V; // uint256[max], task targeted set
    cm; // uint256[2], commitment
}
pp_c; // initialize the parameter of a commitment
pp_a; // initialize the parameter of an accumulator
t_d; // define the duration for refund
Task[] task; // initialize the array of tasks
Task tmpTask; // define a temporary variable
```

A. High-Level Description

We first present our OBFP system from a high-level view as follows. As shown in Fig. 2, to mitigate the burden of storing large amounts of data in smart contract, the user first uploads task data $\mathbf{S}$ into the cloud (Step 1). In Step 2a, the user issues a task to the smart contract. Then, in Step 2b, the worker obtains the task from the smart contract and he/she retrieves the $\mathbf{S}$ from the cloud. Next, he/she computes the task results using it local program and encrypts them based on a one-time key key. In the next round, the worker submits a commitment of key together with an accumulator value of ciphertexts into the smart contract (Step 3a). Here, this submission requires the worker to pay a guarantee deposit υ_g. Also in this round (Step 3b), the worker sends these ciphertexts to the user via "off-chain".

After checking the integrity of received ciphertexts upon the accumulator value stored in the smart contract, the user is supposed to request the decryption key and hence pays the service fee υ_f to the smart contract (Step 4). In order to obtain the υ_f, the worker will reveal the correct key to the smart contract (i.e. response in Step 5). This additionally means that the user can obtain solutions via decrypting the ciphertext and then he/she checks the validation of task results. If there are some solutions $\mathbf{X}' \neq \varnothing$ are incorrect, that is, $\forall x' \in \mathbf{X}'$, $f(x') \notin \mathbf{V}$, then the user can claim for compensation in the sixth round via submitting one of the incorrect ciphertexts and corresponding claim proof. Finally, the worker can finalize and refund its guarantee deposit if the user does not claim before the deadline.

B. Smart Contract

From the above high-level description, we design a smart contract for supporting our OBFP system. It comprises 10 algorithms (namely, OBFP, submitTask, getTask, submitCM, getCM, payRequest, submitProof, getProof, submitCP, and find, see algorithms 1 to 10), where OBFP is a constructor automatically executed when deploying the smart contract, submitTask/getTask is invoked to submit/get a computation task (where getTask is invoked via the targeted user's address, which can be obtained from a bulletin board such as the user's personal website), submitCM/getCM is to submit/get a commitment (where submitCM is a payable function that transfers a guarantee deposit υ_g for its commitment), payRequest is a payable function with paying a service fee υ_f for requesting task solutions, submitProof/getProof is to submit/get a task proof (where the submitProof can get the service fee υ_f if the submitted proof is valid), submitCP is to get the compensation υ_g invoked by the user if the obtained task result is invalid or to refund the υ_g invoked by the worker after a certain locking time, and find is an auxiliary algorithm which locates the place of targeted user identity in the task array.

Here, we set the price of each solution as υ tokens, which can be uniformly determined in the system setup phase. The user needs to pay the worker $\upsilon_f = k\upsilon$ tokens if the number of solutions is k. Correspondingly, the worker must transfer a guarantee deposit $\upsilon_g = 2k\upsilon$ to the smart contract for obtaining the task. The guarantee deposit will be refunded to the worker if its submitted solution set is correct, otherwise be transferred to the user as a compensation. Here, the setting of υ_g is for reference only, other punishment mechanisms (e.g. twice or more charge to repay) can be also adopted.

C. Details of System Design

In this subsection, we integrate a commitment scheme (comprising three algorithms, namely, setup CSetup, commitment CCom, and verification CVerf), an accumulator scheme (setup ASetup, evaluation AEval, witness generation AWit, and verification AVerf), a symmetric encryption scheme (key generation KG, encryption Enc and decryption Dec), and a secure hash function to propose an OBFP system for the outsourcing computation model (denoted in Section III-C).

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 submitTask($f_{des}, h_s, \mathcal{P}_{des}, \mathbf{V}$)

// invoked by user to submit a task.
$tmpTask.user = msg.sender$;
$tmpTask.f_{des} = f_{des}$;
$tmpTask.h_s = h_s$;
$tmpTask.\mathcal{P}_{des} = \mathcal{P}_{des}$;
for each $i \in [0, max)$ **do**
  $tmpTask.\mathbf{V}[i] = \mathbf{V}[i]$;
end for
$task.push(tmpTask)$; // add a new task into the array

Algorithm 3 getTask($user$) View

// invoked by worker to get a task.
$l =$ find($user$); // find the place of $user$ in array
return $(task[l].f_{des}, task[l].h_s, task[l].\mathcal{P}_{des}, task[l].\mathbf{V})$;

Algorithm 4 submitCM($user, k, h, cm$) Payable

// invoked by worker to submit a commitment with transferring a guarantee deposit
$v_g = 2 * k * v$;
require $(msg.value \geq v_g)$;
$l =$ find($user$); // find the place of $user$ in array
if $task[l].worker = null$ **then**
  $task[l].worker = msg.sender$; // binding the worker
  $task[l].k = k$;
  $task[l].h = h$;
  $task[l].cm = cm$;
else
  require $(msg.sender == task[l].worker)$;
  $task[l].k = k$;
  $task[l].h = h$;
  $task[l].cm = cm$;
end if

Algorithm 5 getCM($user$) View

// invoked by user to get a commitment.
$l =$ find($user$); // find the place of $user$ in array
return $(task[l].k, task[l].h, task[l].cm)$;

Algorithm 6 payRequest($user$) Payable

// invoked by user to query solutions with paying a service fee.
$l =$ find($user$); // find the place of $user$ in array
$v_f = task[l].k * v$;
require $((msg.sender == task[l].user)\&\&(msg.value \geq v_f))$;

Specifically, a computation task is denoted as $T = \langle f, \mathbf{S}, \mathbf{V} \rangle$, where the magnitude of task data set $\mathbf{S}$ could be megabytes or more (for which is stored in the cloud) and the size of targeted value set $\mathbf{V}$ is within kilobytes for being directly stored in the smart contract.

Algorithm 7 submitProof($user, key, r$)

// invoked by worker to submit a proof for service fee.
$l =$ find($user$); // find the place of $user$ in array
require$(msg.sender == task[l].worker)$; //check the identity
$(k, h, cm) =$ getCM($task[l].user$);
if (CVerf$(pp_c, cm, key, r) == 1)$ **then**
  $task[l].key = key$;
  $task[l].r = r$;
  $task[l].worker.transfer(v_f)$; //worker gets v_f if valid proof
  $start = now$; // set current block.timestamp as the start time
end if

Algorithm 8 getProof($user$) View

// invoked by user to get a proof.
$l =$ find($user$); // find the place of $user$ in array
require $(msg.sender == task[l].user)$;
return $(task[l].key, task[l].r)$;

Algorithm 9 submitCP($user, d, c_d, \pi_c, scid$)

// invoked by user to submit a claim proof for v_g, or worker to refund v_g after the deadline.
$l =$ find($user$); // find the place of $user$ in array
if $msg.sender == task[l].user$ **then**
  if (AVerf$(pp_a, task[l].h, \{d, c_d\}, \pi_c) == 1)$ **then**
    $x_d =$ Dec$(c_d, task[l].key)$; // use the key to decrypt c
    for each $j \in [0, max)$ **do**
      if $task[l].f_{des}(x_d) == task[l].\mathbf{V}[j]$ **then**
        return $false$;
      end if
    end for
    $task[l].user.transfer(v_g)$;
    return $true$;
  end if
else if $(msg.sender == task[l].worker)\&\&(now > start + t_d)$ **then**
  $task[l].worker.transfer(v_g)$;
  return $true$;
end if

Algorithm 10 find($user$)

// invoked by anyone to find the place of $user$.
for each $l \in [0, task.length)$ **do**
  if $task[l].user == user$ **then**
    return l;
  end if
end for
return $false$; // there exist no $user$ in the array

- $\{pp, scid\} \leftarrow$ Setup(λ): This initialization algorithm takes as input a security parameter λ, and returns some system parameters pp and a smart contract identity $scid$.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

Specifically, it invokes CSetup, ASetup to generate system parameter pp_c and pp_a used in commitment scheme and accumulator scheme, respectively. It also chooses a secure hash function $\mathcal{H}: \{0,1\}^* \rightarrow \{0,1\}^{256}$, and deploys the smart contract described in Section IV-B into the blockchain and obtains an identity $scid$. Thus, the system parameters involved in our OBFP are $pp = \{pp_c, pp_a, \mathcal{H}\}$.

- Issue$(pp, f_{des}, \mathbf{S}, \mathbf{V}, scid)$: A user (e.g. Alice) can invoke this algorithm to issue a computation task by using the system parameter pp, a description f_{des} of function f, a task data set $\mathbf{S}$, a targeted value set $\mathbf{V}$ and the smart contract identity $scid$. First, she computes the hash value of $\mathbf{S}$ (i.e. $h_s = \mathcal{H}(\mathbf{S})$). Then, she uploads the $\mathbf{S}$ to the cloud and obtains the description of index path (denoted as $\mathcal{P}_{des}$) for subsequent retrieving. Finally, she submits the computation task $T = \langle f_{des}, h_s, \mathcal{P}_{des}, \mathbf{V} \rangle$ into the smart contract $scid$.
- Commit$(pp, scid)$: A worker (e.g. Bob), after receiving a task issuing notice [2] from Alice, can invoke this algorithm to commit its answers by using the system parameter pp and smart contract identity $scid$. First, he invokes getTask in the $scid$ to obtain the targeted computation task $T = \langle f_{des}, h_s, \mathcal{P}_{des}, \mathbf{V} \rangle$ and he also uses $\mathcal{P}_{des}$ to query the $\mathbf{S}$ from the cloud. Then, he checks if $h_s \stackrel{?}{=} \mathcal{H}(\mathbf{S})$ (if not, he aborts). Otherwise, he performs its local program to find all x from $\mathbf{S}$ satisfying $f(x) \in \mathbf{V}$. Assuming that the number of such x is k, that is, the solution set is $\mathbf{X} = \{x_1, \ldots, x_k\}$. Next, he invokes KG to generate a encryption key key, and he encrypts the solution set via Enc (i.e. $\mathbf{c} = \{c_1, \ldots, c_k\}$, where $c_i = \mathsf{Enc}(key, x_i), \forall i \in \{1, \ldots, k\}$). He also randomly chooses $r \in \mathbb{Z}_q^*$ to compute a commitment $C = \mathsf{CCom}(pp_c, key, r)$ and an accumulator value $h = \mathsf{AEval}(pp_a, \{i, c_i\}_{i=1}^k)$. Finally, he sends the $\mathbf{c}$ to the user via the "off-chain" method and he submits $\{k, h, C\}$ into the $scid$ via submitCM. The triggering of submitCM requires transferring a guarantee deposit $v_g = 2kv$ as mentioned above.
- Request$(pp, scid)$: The user (i.e. Alice) invokes this algorithm to request the key for decrypting the received ciphertexts. She first invokes the getCM to get $\{k, h, C\}$ from the $scid$. Then, she invokes $\mathsf{AVer}(pp_a, h, \{c_i\}_{i=1}^k)$ to checks if the received ciphertexts are consistent to that promised by the worker. If it does, Alice pays the service fee $v_f = kv$ via triggering the payRequest. Otherwise, she aborts.
- Response$(pp, \pi_r, scid)$: The worker (i.e. Bob) can submit the proof via this algorithm to obtain the service fee v_f. He submits the openness $\pi_r = \{key, r\}$ into the $scid$ via submitProof. The service fee v_f will be automatically transferred to Bob if the openness is valid. Otherwise, the smart contract discards this response. Here, it should be noted that only the accumulator value h, key key, randomness r, and subsequently one invalid ciphertext are public on-chain.
- $\{1, 0\} \leftarrow$ Verify$(pp, \pi_r, scid)$: This algorithm is embedded into the submitProof and automatically performed. After receiving the system parameter pp, the openness $\pi_r = \{key, r\}$ and the smart contract identity $scid$, it first invokes the getCM to get $\{k, h, C\}$ from the $scid$. Then, it invokes $\mathsf{CVerf}(pp_c, key, r, C)$ to check if the $\{key, r\}$ is the openness of the commitment C. If they are, it returns 1 that represents the openness is valid. Otherwise, it returns 0.
- Retrieve$(pp, scid)$: The user (i.e. Alice) uses the system parameter pp and the smart contract identity $scid$ to invoke this algorithm for obtaining the original solution set. She first invokes getProof to obtain $\pi_r = \{key, r\}$ from the smart contract $scid$. Then, she uses the key to decrypt $\mathbf{c}$ for obtaining the original solutions $x_i = \mathsf{Dec}(key, c_i), \forall i \in \{1, \ldots, k\}$.
- Claim$(pp, \mathbf{c}, scid)$: Assuming that the user (i.e. Alice) finds that l solutions $\mathbf{X}' = \{x_{d_1}, \ldots, x_{d_l}\}$ are invalid, that is, $f(x_{d_i}) \notin \mathbf{V}$, where $i \in \{1, \ldots, l\}$. We denote the corresponding ciphertexts as $\mathbf{c}' = \{c_{d_1}, \ldots, c_{d_l}\}$, and Alice randomly chooses one ciphertext $c_{d_j} \in \mathbf{c}'$ to compute the claim proof $\pi_c = \mathsf{AWit}(pp_a, \{i, c_i\}_{i=1, i \neq d_j}^k)$. Note that this proof is generated by Alice who knows the entire ciphertexts $\mathbf{c}$ and therefore can compute π_c. Then, she submits the index d_j, ciphertext c_{d_j} and claim proof π_c into the smart contract $scid$ via submitCP. Alice will obtain the guarantee deposit v_g (paid by the worker, namely, Bob) if the submitCP returns 1. It is worth noting that Alice only needs to recompute the function f on the solutions $\mathbf{X}'$ but not the total task data set $\mathbf{S}$.
- $\{0, 1\} \leftarrow$ VerifyCP$(d, c_d, \pi_c, scid)$: This algorithm is embedded into the Claim and automatically performed. After receiving the system parameter pp, the index of ciphertext d, the claim ciphertext c_d, the claim proof π_c, and the smart contract identity $scid$, this algorithm first invokes getCM to get $\{k, h, C\}$ from the smart contract $scid$. Then, it invokes $\mathsf{AVerf}(pp_a, h, \{d, c_d\}, \pi_c)$ to verify the correctness of accumulator witness π_c. If the π_c is correct, it invokes getProof to obtain $\{key, r\}$ from the $scid$, and then it uses the key to compute $x = \mathsf{Dec}(key, c_d)$ via Dec. Finally, it checks if the conditional $f(x) \notin \mathbf{V}$ holds on or not. If it does, it returns 1 meaning that the claim proof is valid. Otherwise, it returns 0.

On basis of the properties mentioned in [10], [12], [14] (i.e. "completeness", "robust fairness", "compatibility"), we distinguish "soundness" from the "completeness" to further ensure system reliability. In addition, a blockchain-based fair payment for outsourcing computations should also satisfy the "pay-as-you-go" for utility.

- *Completeness:* If a response is correctly generated by Response, then it must be accepted (i.e. Verify returns 1). Analogously, as a reasonable claim proof generated by Claim, the VerifyCP will return 1.

[2]Due to the easily ignored or missed *event* tool (e.g. Ethereum *event*, available at: https://docs.soliditylang.org/en/v0.8.2/abi-spec.html#events), we suggest using a bulletin board such as the user's personal website to realize the notice. Thus, the worker can autonomously obtain the notice (including the user's address and brief description of tasks) from the user's personal website.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

- *Soundness:* This property corresponds to the above correctness, which emphasizes that incorrect responses or claim proofs must be rejected.
- *Robust Fairness:* The fairness is twofold in our OBFP system. On the one hand, the fairness for worker (or user) represents that it is impossible for a malicious user (resp. worker) to enjoy valid task response (resp. submit task commitment) without paying enough service fees (resp. guarantee deposits). On the other hand, the malicious worker will be punished if providing incorrect responses to obtain the service fee, i.e., the user will get the guarantee deposit paid by the worker. Obviously, the latter can be guaranteed by the aforementioned correctness.
- *Compatibility:* An OBFP should be compatible with different types of blockchain to reach a wide range of applications in different scenarios such as the Ethereum blockchain in public and the Hyperledger Fabric blockchain in permissioned.
- *Pay-As-You-Go*. This property guarantees convenience and utilitarian nature of an OBFP. That is, a worker can obtain the service fee if providing valid computation responses, without the collaboration of a user or waiting for the deadline. Correspondingly, a user can provide a claim proof for compensation. If the claim proof is valid, the user will directly get the worker's guarantee deposit without requiring any confirmation from the worker or waiting for deadline.

Theorem 1: The tuple $\Pi = (\mathsf{Setup}, \mathsf{Issue}, \mathsf{Commit}, \mathsf{Request}, \mathsf{Response}, \mathsf{Verify}, \mathsf{Retrieve}, \mathsf{Claim}, \mathsf{VerifyCP})$ is a secure OBFP system, that is, it satisfies "Completeness", "Soundness", "Robust Fairness", "Compatibility" and "Pay-As-You-Go".

Proof: To prove the "*Completeness*", assuming that $\{k, h, C\}$ is a commitment submitted by $\mathsf{submitCM}$, and $\pi_r = \{key, r\}$ a response generated by $\mathsf{Response}$, we have $\mathsf{CVerf}(pp_c, key, r, C) = 1$. This means that the $\mathsf{Response}$ is perfectly completed with Verify. In addition, given a claim π_c generated by Claim, we can invoke $\mathsf{submitCM}$ to obtain $\{k, h, C\}$ and we have $\mathsf{AVerf}(pp_a, h, \{d, c_d\}, \pi_c) = 1$. This implies the consistency between Claim and $\mathsf{VerifyCP}$.

The "*Soundness*" follows from the security properties of a secure commitment (i.e. satisfying the properties of binding and hiding) and the collusion resistance of a secure hash function. For the soundness of worker, there are two cases. On the one hand, we suppose a $\mathcal{PPT}$ worker W^* has ever generated the commitment $C = \mathsf{CCom}(pp_c, key, r)$ which was submitted into the smart contract. However, he/she plans to obtain the service fee without revealing the real key. Hence, he/she generates an accepted proof $\pi_r^* = (key^*, r^*)$, but the key^* is an incorrect key for decrypting $\mathbf{c}$. That is, the equation $\mathsf{CVerf}(pp_c, key^*, r^*, C) = 1$ holds on but $\mathsf{Dec}(key^*, c_i) \neq x_i, \forall i \in \{1, \ldots, k\}$. This will violate the binding of adopted commitment scheme, for the W^* has found two different (key, r) and (key^*, r^*) such that $\mathsf{CCom}(key^*, r^*) = \mathsf{CCom}(key, r) = C$. On the other hand, the W^* has never generated the $C = \mathsf{CCom}(key, r)$, but it successfully submits $\pi_r = \{key, r\}$ to get the service fee. This probability is negligible for the hiding of adopted commitment scheme. Note that, even the W^* successes by a fluke, it will lose its guarantee deposit due to the claim of user. This shows that our OBFP actually provides a double soundness.

For the soundness of user, on the one hand, we suppose a $\mathcal{PPT}$ user U^* successfully obtains the guarantee deposit with lying. That is, it forges an accepted $\{d, c_d^*, \pi_c\}$ such that $\mathsf{AVerf}(pp_a, h, \{d, c_d^*\}, \pi_c) = 1$ and $\mathsf{Dec}(key, c_d^*) \notin \mathbf{V}$. However, the original dth ciphertext from honest worker is actually correct. It means that the U^* has found another ciphertext $c_d^* \notin \{c_i\}_{i=1}^k$ to compute a valid π_c such that $\mathsf{AVerf}(pp_a, h, \{d, c_d^*\}, \pi_c) = 1$, and submits the invalid c_d^* for obtaining the compensation. This is not allowed for the collusion resistance of a secure accumulator scheme.

To prove the "*Robust Fairness*", we assume that there exists a malicious worker W' (or user U') that could successfully break the fairness with a non-negligible probability. This means that W' (resp. U') can break the constraint of smart contract (i.e. bypassing the verification of $\mathsf{require}$ function [3]). This will contradict the security of our designed smart contract, especially we propose using finite state machine (FSM) based model [28] together with a security testing tool (e.g. [29], [30]) for designing our secure smart contract. Moreover, the setting of service fee and guarantee deposit further guarantees that both the user and worker do not choose to trick for obtaining maximum profits. Finally, it is obvious that our proposal does not involve any TTP during the process of trading. Thus, our proposed OBFP satisfies the property of robust fairness. Someone may argue that, in the Setup, some parameters such as $\{pp_c, pp_a\}$ may be generated by some trusted party. Here, we propose using the existing decentralized generation method (e.g. [31], [32]) to ensure decentralization.

Considering the "*Compatibility*", we should take the functionality of smart contract into account when deploying, for which is the core component. Obviously, our OBFP is suitable for those blockchain systems (both public and permissioned ones) supporting smart contract, for example, Ethereum and Hyperledger Fabric. Despite in some blockchain systems that executing computation or storing data will cause costs (e.g. gas costs in Ethereum), our OBFP can still support well for which does not involve expensive operations. One challenge of compatibility is that the computation task may involve a mass of data, but we have adopted the cloud to store large task data (i.e. the aforementioned $\mathbf{S}$), and only the targeted value data of small size (i.e. $\mathbf{V}$) are submitted into the smart contract. This can efficiently ease the storage burden of blockchain, and hence guarantees the compatibility of our OBFP. The other challenge is to provide the trading function for some permissioned blockchain systems (e.g. Hyperledger Fabric) which have removed the concept of token [33]. Nevertheless, some existent token projects can be directly integrated such as fabric-token.[4]

The final "*Pay-As-You-Go*" is achieved by the commitment scheme, accumulator scheme, hash function and the automatic

[3]The $\mathsf{require}$ is a convenient function for checking if conditions hold or not. If they do not, it will throw an exception.

[4]Fabric-token, available at: https://github.com/vadimuvin/fabric-token

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

execution of smart contract. Concretely, in the Response, a worker can submit its response (i.e. openness) via submitResponse that is automatically validated by Verify. If the response is valid, it will obtain the service fee prepaid by the user. Moreover, after receiving the response submitted by the worker, the user can use it to decrypt its received ciphertext, and further verifies if the original plaintext is really the solution to its issued task. If it is not, the user can claim for compensation by submitting a claim proof via submitCP in Claim. Likewise, VerifyCP is also automatically executed for checking the input claim proof, and the compensation (i.e. the paid guarantee deposit by the worker) will be obtained by the user if the proof is valid. These two processes do not require confirmation from the other party or waiting for a preset deadline. Maybe the only deadline is that the system will automatically refund guarantee deposit to the worker if the user does not claim on due.

V. Application and Performance Evaluation

The efficiency of our OBFP system can vary for different tasks $f = \langle f, \mathbf{S}, \mathbf{V} \rangle$, especially be influenced by the final solutions $\mathbf{X} \subset \mathbf{S}$ and targeted value set $\mathbf{V}$. Some scenarios with small $\mathbf{X}$ and $\mathbf{V}$ are the most promising candidates. In such scenarios, the fee for storing data in the smart contract is small and the contract triggering can be efficient. In this section, we will show how to apply our OBFP into security evaluation of authentication schemes. The outsourcing computation task here is searching the preimage of some hash values from some open dataset (i.e. launching an offline dictionary guess on authentication schemes).

More specifically, we apply our OBFP system into the offline dictionary guess on Hsieh et al.'s scheme [34] proposed by Wang *et al.* [16]. Also, we further provide performance analysis of our OBFP including cloud scalability and concurrency, gas cost, communication cost and computation cost by using the commitment scheme (i.e. Pedersen commitment [23]), the accumulator scheme (i.e. RSA-based accumulator [31]), the symmetric encryption scheme (i.e. an IND-CPA secure scheme [14]) and the hash function (i.e. Keccak-256).

A. Applied Case

Before describing the application of our OBFP system, we briefly review the proposed offline dictionary guess in [34]. During the login phase, a user uses its smart card to compute $P_1 = R \oplus \mathcal{H}(b \oplus PW_u)$ and $P_2 = \mathcal{H}(P_1 \oplus T_u)$, where randomnesses R, b and cryptographic hash function $\mathcal{H}$ are stored in the smart card. Wang *et al.* [16] have pointed out that an offline dictionary guess attack on this login phase can be successfully launched as follows.

1) Suppose the value of PW_i is set as PW_i^* from the dictionary set $\mathcal{S}_{pw}$.
2) Compute $P_1^* = R \oplus \mathcal{H}(b \oplus PW_i)$ and $P_2^* = \mathcal{H}(P_1^* \oplus T_i)$, where R, b is extracted from the smart card, T_i is intercepted from the public channel.
3) Validate PW_i^* via checking whether the computed P_2^* is equal to the intercepted P_2.
4) Repeat the above Step 1) to 3) to find the correct PW_i.

To apply our OBFP system into the above case, we set the computation task as $T = \langle f, \mathbf{S}, \mathbf{V} \rangle$, where $f(x) = \mathcal{H}(R \oplus \mathcal{H}(b \oplus x) \oplus T_i)$, $\mathbf{S} = \mathcal{S}_{pw}$, $\mathbf{V} = \{R, b, \mathbf{T}, \mathbf{P}_2\}$. The corresponding goal is to find $PW_i \in \mathbf{S}$, s.t., $f(PW_i) = \mathcal{H}(R \oplus \mathcal{H}(b \oplus PW_i) \oplus T_i) = P_{2,i}$, where $T_i \in \mathbf{T}$, $P_{2,i} \in \mathbf{P}_2$.

B. Implementation

To evaluate the performance of our OBFP for the above applied case, we first evaluate the cloud storage and inbound/outbound traffic. Concretely, the COSBench [5] is used in our evaluation with setting authentication type as "mock", storage type as "mock", object size as "6400 bytes", operation types as "read" and "write" (where the ratio of them is 80% and 20%, respectively), running time as "120 seconds". The test code could be saw in the website ,[6] and the system configuration is the Ubuntu system (16.04 version, 64 bits) with an Inter(R) Core(TM) i7-9750H CPU of 2.60 GHz and 2-GB RAM.

We then use Remix[7] to implement the prototype of our smart contract. The concrete configuration is the compiler (0.4.19+commit.c4cbbb05), language (Solidity), EVM version (compiler default), Deployment Environment (JavaScript Virtual Machine) and Featured Plugins (Debugger, Deploy and Run Transactions, Solidity Compiler, Solidity Static Analysis, and Solidity Unit Testing).

Before deploying our smart contract, we need to decide which elliptic curve to use, for our OBFP requires additive group operations in non-singular elliptic curve. Here, we adopt the cheaper (about gas cost) curve alt_bn128 (which was also chosen in realizing zkSNARKs). Specifically, we use the precompiled contracts supporting point addition and scale multiplication on the alt_bn128[8] for our design. Then, we deploy the smart contract[9] in the Remix to test the gas costs of every function.

Furthermore, we analyze the efficiency (in terms of communication and computation costs) from perspectives of user, worker and smart contract. Here, we choose FairSwap and OptiSwap in the comparative analysis, for which are two of the most recent constructions with efficiency and similar architecture to our proposal. Specifically, we adopt the public key of RSA system as 2048 bits and Barreto-Naehrig (BN) over base $\mathbb{F}$-256 for achieving the approximate security level $\lambda = 112$. The notations and corresponding size/time are shown in Table I, where the experiment is conducted in our personal computer (PC) upon a popular cryptographic library, namely, miracl library. The system configuration of our PC is the Windows system (Windows 7, 64 bits) with an Inter(R) Core(TM) i5-4210U CPU of 1.70 GHz and 4-GB RAM.

[5]COSBench is a benchmarking tool to measure of performance of Cloud Object Storage services, available at: https://github.com/intel-cloud/cosbench

[6]Our COSBench test code is available at: https://github.com/colyn91/OBFP/blob/master/obfp-config.xml

[7]A Browser Solidity providing IDE for compiling and deploying Solidity smart contract. Link: http://remix.ethereum.org.

[8]https://github.com/ethereum/EIPs/pull/213

[9]The sourcecode of our smart contract could be found at: https://github.com/colyn91/OBFP

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS, DESCRIPTIONS AND SIZE/TIME

Notation	Description	Size (byte)	Notation	Description	Time (millisecond)
S_s	Size of type $string$	32	T_{enc}	Time of encryption algorithm	0.0006
S_u	Size of type $uint256$	32	T_{dec}	Time of decryption algorithm	0.0006
$S_{\mathbf{V}}$	Size of targeted value set $\mathbf{V}$	2048	T_{pa}	Time of point addition in $\mathbb{G}$	0.0811
S_n	Size of an element in $\mathbb{Z}_n^*$	128	T_{sm}	Time of scale multiplication in $\mathbb{G}$	8.8517
			T_h	Time of hash function $keccak256$	0.0006
			T_{mm}	Time of modular multiplication in $\mathbb{Z}_n^*$	0.0108
			T_{me}	Time of modular exponentiation in $\mathbb{Z}_n^*$	2.7072

*Note that the setting of size of $string$ and $\mathbf{V}$ varies from different applications.

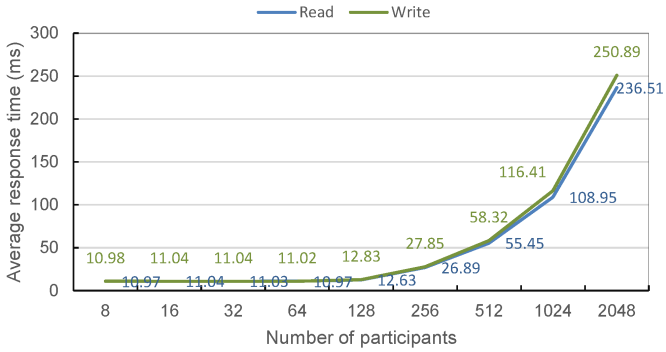


Fig. 3. Average response time in mock type of cloud storage.

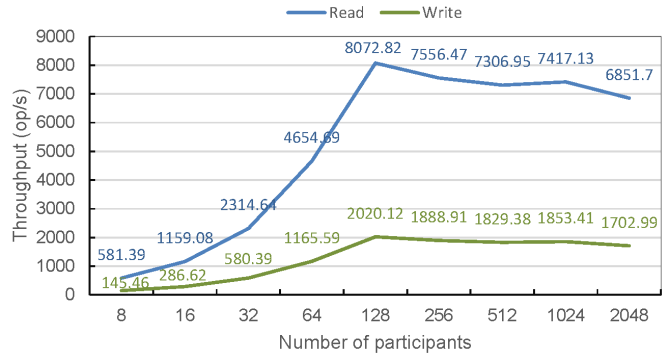


Fig. 4. Throughput in mock type of cloud storage.

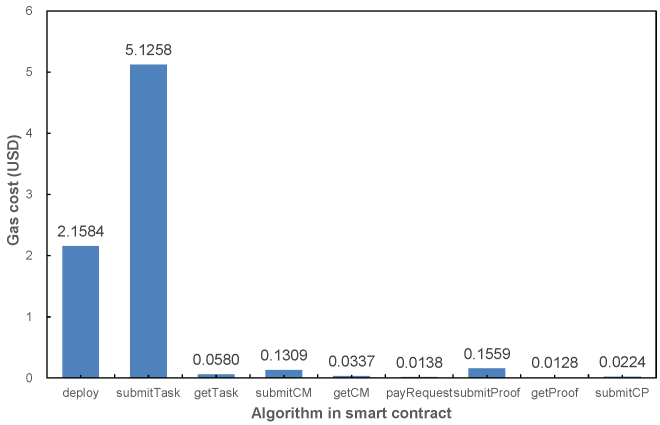


Fig. 5. Smart contract gas cost in our OBFP.

C. Benchmarks

Cloud Scalability and Concurrency: The evaluation results are shown in Figure 3 and 4, where the number of participants increases from 8 to 2048 with an exponential growth. We can find that the average response time almost keeps unchanged (about 12 milliseconds) if the number of participants is less than 128, but after that it increases rapidly. Correspondingly, the throughput of "read" and "write" reaches to the peak value (i.e. 8072.82 op/s and 2020.12 op/s respectively, where the "op" is the abbreviation of "operation") when the number of participants is 128. This means that the scalability and concurrency in cloud storage are closely related to the number of participants, and the optimal scale or parallel quantity is about 128. Such a scale can efficiently support our OBFP to outsource computing tasks from one user to multiple workers.

Gas Cost: The costs (exactly gas costs) of running our OBFP will cause fees, which are paid to the miners of cryptocurrency (i.e. Ethereum in our prototype). We set the Gas Limit as $10,000,000$ gas, which for a gas price of 2 $GWei$ translates to 0.006 $Ether$ or 1.35 USD for an exchange rate of 225 USD per $Ether$.[10] As shown in Fig. 5, the gas cost for triggering the functions Deploy, submitTask, getTask, submitCM, getCM, payRequest, submitProof, getProof, submitCP remains almost constant for different parameters. This means that both the user and worker do not need to pay more fees in terms of gas costs even involving more task solutions. Deploying smart contract and invoking submitTask cost more gas costs (i.e. 2.1584 USD and 5.1258 USD, respectively), nevertheless, these two operations only need to be executed once. Every left operation costs no more than 16 US cents, which is really the advantage of our OBFP comparing to FairSwap [14] and OptiSwap [11]. Especially in both FairSwap and OptiSwap, the gas cost of claim phase varies highly, depending on the choice of file chunk size and merkle tree depth.

Communication and Computation Costs: To demonstrate the major benefit of our OBFP system, we also compare it with FairSwap and OptiSwap in terms of communication and computation costs. The theoretical and practical results of communication and time costs are listed in Table II, where the notations k, m, l_g represent the numbers of solutions, circuit gates and the number of each gate's inputs, respectively [11], [14].

In the theoretical analysis, we first count the cryptographic operations of each algorithm in OBFP. The communication

[10]The exchange rate is referred to CoinMarketCap on 7 February, 2020, available at: https://coinmarketcap.com/

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMMUNICATION AND COMPUTATION COSTS OF OUR OBFP, FAIRSWAP [14] AND OPTISWAP [11]

		User			Worker			Smart Contract		
		Our OBFP	FairSwap	OptiSwap	Our OBFP	FairSwap	OptiSwap	Our OBFP	FairSwap	OptiSwap
Theory	Comm. cost	$S_s + S_{\mathbf{V}}$ $+9S_u + 2S_n$	$S_s + S_{\mathbf{V}} + 9S_u +$ $l_g \log(k+m)S_u$ $+2\log(k+m)S_u$	$S_s + S_{\mathbf{V}} + 15S_u +$ $\log(k+m)S_u$	$S_s + S_{\mathbf{V}}$ $+6S_u + S_n$	$S_s + S_{\mathbf{V}}$ $+9S_u$	$S_s + S_{\mathbf{V}}$ $+17S_u$	No	No	No
	Time cost	$kT_{dec} + 2T_{me}$ $+(2k+1)T_h$ $+(2k-3)T_{mm}$	$l_g \log(k+m)T_h$ $+6\log(k+m)T_h$ $+(k+m)T_{dec}$	$l_g \log(k+m)T_h$ $+\log(k+m)T_h$ $+(k+m)T_{dec}$	$kT_{enc} + T_{pa} +$ $(k+1)T_h + 2T_{sm} +$ $T_{me} + (k-1)T_{mm}$	$2T_{sm} + T_{pa} +$ $(k+m)T_{enc} +$ $2\log(k+m)T_h$	$2T_{sm} + T_{pa} +$ $(k+m)T_{enc}$ $+\log(k+m)T_h$	$2T_{sm} + T_{pa}$ $+T_h + T_{me}$ $+T_{dec}$	$2T_{sm} + T_{pa} +$ $(l_g+1)T_{dec} +$ $l_g \log(k+m)T_h$ $+2\log(k+m)T_h$	$2T_{sm} + T_{pa} +$ $\log(k+m)T_h$
Practice	Comm. cost (byte)	2624	$2368 + 64\log(k+m)$ $+32l_g \log(k+m)$	$2560 + 32\log(k+m)$	2400	2368	2624	0	0	0
	Time cost (millisecond)	$0.0234k+$ 5.3826	$0.0006(k+m)+$ $0.0006l_g \log(k+m)$ $+0.0036\log(k+m)$	$0.0006\log(k+m)+$ $0.0006l_g \log(k+m)$ $+0.0006(k+m)$	$0.012k + 20.4815$	$17.7845+$ $0.0012\log(k+m)$ $+0.0006(k+m)$	$17.7845+$ $0.0006\log(k+m)$ $+0.0006(k+m)$	20.4929	$17.7851 + 0.0006l_g+$ $0.0006l_g \log(k+m)$ $+0.0012\log(k+m)$	$17.7845+$ $0.0006\log(k+m)$

*The notations k, m, l_g represent the numbers of solutions, circuit gates and the number of each gate's inputs, respectively [11], [14].

costs of user and worker are $S_s + S_{\mathbf{V}} + 9S_u + 2S_n$ and $S_s + S_{\mathbf{V}} + 6S_u + S_n$ respectively, and the computation costs of user, worker and smart contract are $kT_{dec} + 2T_{me} + (2k+1)T_h + (2k-3)T_{mm}$, $kT_{enc} + T_{pa} + (k+1)T_h + 2T_{sm} + T_{me} + (k-1)T_{mm}$ and $2T_{sm} + T_{pa} + T_h + T_{me} + T_{dec}$ respectively. Then, we count the communication and computation costs in FairSwap, namely, the former costs are $S_s + S_{\mathbf{V}} + 9S_u + l_g \log(k+m)S_u + 2\log(k+m)S_u$ and $S_s + S_{\mathbf{V}} + 9S_u$ for user and worker respectively, and the latter costs are $l_g \log(k+m)T_h + 6\log(k+m)T_h + (k+m)T_{dec}$, $2T_{sm} + T_{pa} + (k+m)T_{enc} + 2\log(k+m)T_h$ and $2T_{sm} + T_{pa} + (l_g+1)T_{dec} + l_g \log(k+m)T_h + 2\log(k+m)T_h$ for user, worker and smart contract respectively. Also in OptiSwap, the communication costs are $S_s + S_{\mathbf{V}} + 15S_u + \log(k+m)S_u$, $S_s + S_{\mathbf{V}} + 17S_u$ for user and worker respectively, and the computation costs are $l_g \log(k+m)T_h + \log(k+m)T_h + (k+m)T_{dec}$, $2T_{sm} + T_{pa} + (k+m)T_{enc} + \log(k+m)T_h$ and $2T_{sm} + T_{pa} + \log(k+m)T_h$ for user, worker and smart contract respectively. Notably, the smart contract directly retrieves data from the local database and thus causes no communication cost.

On basis of the experiment results listed in Table I, we can obtain the practical results of communication and time costs. In our OBFP system, the former costs of user and worker are 2624 bytes and 2400 bytes respectively, and the latter costs of user, worker and smart contract are $0.0234k + 5.3826$ milliseconds, $0.012k + 20.4815$ milliseconds and 20.4929 milliseconds respectively. In the FairSwap [14], the practical communication costs of user and worker are $2368 + 64\log(k+m) + 32l_g \log(k+m)$ bytes and 2368 respectively, and the practical computation costs of user, worker and smart contract are $0.0006(k+m) + 0.0006l_g \log(k+m) + 0.0036\log(k+m)$, $17.7845 + 0.0012\log(k+m) + 0.0006(k+m)$, and $17.7851 + 0.0006l_g + 0.0006l_g \log(k+m) + 0.0012\log(k+m)$ respectively. Correspondingly, in OptiSwap, the practical communication costs are $2560 + 32\log(k+m)$, 2624 for user and worker respectively, and the practical computation costs are $0.0006\log(k+m) + 0.0006l_g \log(k+m) + 0.0006(k+m)$, $17.7845 + 0.0006\log(k+m) + 0.0006(k+m)$ and $17.7845 + 0.0006\log(k+m)$ for user, worker and smart contract respectively.

We observe that the communication cost of worker in OBFP has been largely reduced by comparing to that in FairSwap and OptiSwap. Specifically, the communication cost of worker in FairSwap and OptiSwap will exceed that in OBFP once $k + m \geqslant 4$ with setting the number of circuit gate' inputs as $l_g = 2$, for the latter is a constant (i.e. 2400 bytes). The user in OBFP has to consume an additional communication cost (about 32 bytes) than FairSwap, but saves about 224 bytes than OptiSwap. As the computation cost, both the user and worker in OBFP, FairSwap and OptiSwap have to execute more cryptographic operations as the size of solutions increasing. While the growth rate of computation cost in OBFP (in terms of user and worker) is more than that in FairSwap and OptiSwap, smart contract in OBFP only needs to execute some constant cryptographic operations (about 20.4929 milliseconds). This is really an advantage of our proposed OBFP system when smart contracts are involved for practical implementation.

Significantly, OptiSwap has included SmartJudge [35] for further improvements, where the SmartJudge is more like a plug-in when being integrated into OptiSwap. Thus, for further improving the efficiency, we can also integrate the SmartJudge into our OBFP system via submitting hash value of related negotiated terms (e.g. commitment, proof, decryption key) and transmitting these terms off-chain. This will further show the advantage of our OBFP system.

Transaction Number: It is well known that the bottleneck of current blockchain systems is the transaction delay. To evaluate feasibility of our OBFP, we further count the transaction number and find that it is always a constant 5 even though the size of solutions keeps increasing. This means that the total time cost in our OBFP (implemented in Ethereum) is about one minute, for the time of generating one block in Ethereum is about 12 seconds. This, from another point of view, means that the increasing time cost of cryptographic operations in our smart contract will not impact significantly. We propose using other blockchain systems with less delay such as Hyperledger Fabric[11] or constructing a specialized blockchain platform to further reduce this time cost.

D. Comparison

We finally compare our OBFP with some existing fair payment systems to illustrate the security and utility of our proposal. We denote "Y" and "N" as supported and unsupported feature, respectively. The "Optimistic" means that still requiring a TTP when involving a dispute, and "Robust$^-$" represents that achieving fairness without any TTP but somewhat compromises the worker's fairness. The "Const/Non-const" refers to the transaction number in the system is constant or

[11]The latency of Hyperledger Fabric is less than one second, which is available at https://www.hyperledger.org/

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF KEY FEATURES WITH THE EXISTING SYSTEMS

System	No Bank	Fairness	Compatibility	Pay-As-You-Go	ZKP-Free	Transaction Number
Chen et al. [1]	N	Optimistic	N	N	N	Const
Huang et al. [10]	Y	Optimistic	N	N	Y	Const
Campanelli et al. [13]	Y	Robust	Y	Y	N	Const
Dziembowski et al. [14]	Y	Robust$^-$	Y	N	Y	Const
Zhang et al. [12]	Y	Robust$^-$	Y	N	Y	Non-const
Zhang et al. [15]	Y	Robust$^-$	Y	N	Y	Non-const
Eckey et al. [11]	Y	Robust$^-$	Y	N	Y	Const
Our OBFP	Y	Robust	Y	Y	Y	Const

not. From the comparison results in Table III, one can find that our OBFP owns more advantage for the application in outsourcing computations.

VI. CONCLUSION

In this paper, we focus on designing an optimized blockchain-based fair payment (OBFP) system for outsourcing computations. We first present the system model of OBFP and then we construct a novel solution upon blockchain and cryptographic tools (i.e. Pedersen commitment [23], RSA-based accumulator [31], the IND-CPA secure symmetric encryption scheme [14] and the Keccak-256 hash function). Moreover, we show that our OBFP satisfies the security properties including completeness, soundness, robust fairness, compatibility, and pay-as-you-go. To show its utility, we further instantiate it into the offline dictionary password guess outsourcing, together with implementing it via the COSBench and Remix. From the final benchmarks and comparison, we can conclude that our OBFP works more fair with a lower executing cost for being adopted in the outsourcing computations.

REFERENCES

[1] X. Chen, J. Li, and W. Susilo, "Efficient fair conditional payments for outsourcing computations," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 6, pp. 1687–1694, Dec. 2012.

[2] B. Carbunar and M. V. Tripunitara, "Payments for outsourced computations," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 2, pp. 313–320, Feb. 2012.

[3] B. Carbunar and M. V. Tripunitara, "Conditional payments for computing markets," in *Proc. 7th Int. Conf. Cryptol. Netw. Secur. (CANS)*, vol. 5339, M. K. Franklin, L. C. K. Hui, and D. S. Wong, Eds. Hong Kong: Springer, 2008, pp. 317–331.

[4] B. Carbunar and M. Tripunitara, "Fair payments for outsourced computations," in *Proc. 7th Annu. IEEE Commun. Soc. Conf. Sensor, Mesh Ad Hoc Commun. Netw. (SECON)*, Jun. 2010, pp. 529–537.

[5] P. Golle and I. Mironov, "Uncheatable distributed computations," in *Proc. Cryptographers Track RSA Conf.*, 2001, pp. 425–440.

[6] N. Asokan, M. Schunter, and M. Waidner, "Optimistic protocols for fair exchange," in *Proc. 4th ACM Conf. Comput. Commun. Secur. (CCS)*, Apr. 1997, pp. 7–17.

[7] Q. Huang, G. Yang, D. S. Wong, and W. Susilo, "Ambiguous optimistic fair exchange," in *Advances in Cryptology—ASIACRYPT* (Lecture Notes in Computer Science), vol. 5350, J. Pieprzyk, Ed. Melbourne, VIC, Australia: Springer, Dec. 2008, pp. 74–89.

[8] M. T. Dashti, "Efficiency of optimistic fair exchange using trusted devices," *ACM Trans. Auto. Adapt. Syst.*, vol. 7, no. 1, pp. 1–18, Apr. 2012.

[9] Q. Huang, G. Yang, D. S. Wong, and W. Susilo, "Ambiguous optimistic fair exchange: Definition and constructions," *Theor. Comput. Sci.*, vol. 562, pp. 177–193, Jan. 2015.

[10] H. Huang, X. Chen, Q. Wu, X. Huang, and J. Shen, "Bitcoin-based fair payments for outsourcing computations of fog devices," *Future Gener. Comput. Syst.*, vol. 78, pp. 850–858, Jan. 2018.

[11] L. Eckey, S. Faust, and B. Schlosser, "OptiSwap: Fast optimistic fair exchange," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 543–557.

[12] Y. Zhang, R. Deng, X. Liu, and D. Zheng, "Outsourcing service fair payment based on blockchain and its applications in cloud computing," *IEEE Trans. Services Comput.*, early access, Aug. 7, 2018, doi: 10.1109/TSC.2018.2864191.

[13] M. Campanelli, R. Gennaro, S. Goldfeder, and L. Nizzardo, "Zero-knowledge contingent payments revisited: Attacks and payments for services," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 229–243.

[14] S. Dziembowski, L. Eckey, and S. Faust, "FairSwap: How to fairly exchange digital goods," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 967–984.

[15] Y. Zhang, R. H. Deng, X. Liu, and D. Zheng, "Blockchain based efficient and robust fair payment for outsourcing services in cloud computing," *Inf. Sci.*, vol. 462, pp. 262–277, Jun. 2018.

[16] D. Wang and P. Wang, "Offline dictionary attack on password authentication schemes using smart cards," in *Proc. 16th Int. Conf. Inf. Secur. (ISC)*, vol. 7807, 2013, pp. 221–237.

[17] W. Du, J. Jia, M. Mangal, and M. Murugesan, "Uncheatable grid computing," in *Proc. 24th Int. Conf. Distrib. Comput. Syst.*, Mar. 2004, pp. 4–11.

[18] C. Lin, D. He, X. Huang, M. K. Khan, and K.-K.-R. Choo, "DCAP: A secure and efficient decentralized conditional anonymous payment system based on blockchain," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2440–2452, 2020.

[19] A. Jindal, G. S. Aujla, and N. Kumar, "SURVIVOR: A blockchain based edge-as-a-service framework for secure energy trading in SDN-enabled vehicle-to-grid environment," *Comput. Netw.*, vol. 153, pp. 36–48, Apr. 2019.

[20] R. Chaudhary, A. Jindal, G. S. Aujla, S. Aggarwal, N. Kumar, and K.-K.-R. Choo, "BEST: Blockchain-based secure energy trading in SDN-enabled intelligent transportation system," *Comput. Secur.*, vol. 85, pp. 288–299, Aug. 2019.

[21] M. Al-Bassam, A. Sonnino, M. Król, and I. Psaras, "Airtnt: Fair exchange payment for outsourced secure enclave computations," 2018, *arXiv:1805.06411*. [Online]. Available: https://arxiv.org/abs/1805.06411

[22] H. Dang, D. L. Tien, and E. Chang, "Fair marketplace for secure outsourced computations," 2018, *arXiv:1808.09682*. [Online]. Available: https://arxiv.org/abs/1808.09682

[23] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO 11th Annu. Int. Cryptol. Conf. Adv. Cryptol.*, 1991, pp. 129–140.

[24] J. Benaloh and M. de Mare, "One-way accumulators: A decentralized alternative to digital signatures," in *Proc. EUROCRYPT Workshop Theory Appl. Cryptograph. Techn. Adv. Cryptol.*, 1994, pp. 274–285.

[25] N. Fazio and A. Nicolosi, "Cryptographic accumulators: Definitions, constructions and applications," in *Paper Written for Course at New York University*. New York, NY, USA, 2003, pp. 1–23, [Online]. Available: www.cs.nyu.edu/nicolosi/papers/accumulators.pdf

[26] S. Aggarwal, R. Chaudhary, G. S. Aujla, N. Kumar, K.-K.-R. Choo, and A. Y. Zomaya, "Blockchain for smart communities: Applications, challenges and opportunities," *J. Netw. Comput. Appl.*, vol. 144, pp. 13–48, Oct. 2019.

[27] A. Miglani, N. Kumar, V. Chamola, and S. Zeadally, "Blockchain for Internet of energy management: Review, solutions, and challenges," *Comput. Commun.*, vol. 151, pp. 395–418, Feb. 2020.

[28] A. Mavridou and A. Laszka, "Designing secure ethereumphsmart contracts: A finite state machine based approach," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2018, pp. 523–540.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.

[29] R. M. Parizi, A. Dehghantanha, K.-K. R. Choo, and A. Singh, "Empirical vulnerability analysis of automated smart contracts security testing on blockchains," in *Proc. CASCON 28th Annu. Int. Conf. Comput. Sci. Softw. Eng.*, 2018, pp. 103–113.
[30] R. M. Parizi, Amritraj, and A. Dehghantanha, "Smart contract programming languages on blockchains: An empirical evaluation of usability and security," in *Proc. Int. Conf. Blockchain*, 2018, pp. 75–91.
[31] L. Xu, L. Chen, Z. Gao, S. Xu, and W. Shi, "Efficient public blockchain client for lightweight users," *ICST Trans. Secur. Saf.*, vol. 4, no. 13, Jan. 2018, Art. no. 153528.
[32] B. Bunz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 315–334.
[33] E. Androulaki *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, Apr. 2018, p. 30.
[34] W.-B. Hsieh and J.-S. Leu, "Exploiting hash functions to intensify the remote user authentication scheme," *Comput. Secur.*, vol. 31, no. 6, pp. 791–798, Sep. 2012.
[35] E. Wagner, A. Volker, F. Fuhrmann, R. Matzutt, and K. Wehrle, "Dispute resolution for smart contract-based two-party protocols," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2019, pp. 422–430.



Chao Lin received the Ph.D. degree from the School of Cyber Science and Engineering, Wuhan University, in 2020. He currently works with the College of Mathematics and Informatics, Fujian Normal University, China. His research interests mainly include applied cryptography and blockchain technology.



Debiao He (Member, IEEE) received the degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He has published over 150 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and Usenix Security Symposium. His main research interests include cryptography and information security, in particular, cryptographic protocols. He was a recipient of the 2018 IEEE SYSTEMS JOURNAL Best Paper Award and the 2019 IET Information Security Best Paper Award. His work has been cited more than 9000 times at Google Scholar. He is in the Editorial Board of several international journals, such as *Journal of Information Security and Applications*, *Frontiers of Computer Science*, and *Human-centric Computing and Information Sciences*.



Xinyi Huang received the Ph.D. degree from the School of Computer Science and Software Engineering, University of Wollongong, Australia. He is currently a Professor with the School of Mathematics and Computer Science, Fujian Normal University, China, and the Co-Director of the Fujian Provincial Key Laboratory of Network Security and Cryptology. He has authored over 100 research papers in refereed international conferences and journals. His research interests include applied cryptography and network security. His work has been cited more than 9000 times at Google Scholar (H-Index: 50). He is also an Associate Editor of the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the Ph.D. degree in information security from the Queensland University of Technology, Australia, in 2006. He currently holds the Cloud Technology Endowed Professorship at The University of Texas at San Antonio. He is also an ACM Distinguished Speaker and the IEEE Computer Society Distinguished Visitor from 2021 to 2023, and included in the Web of Science's Highly Cited Researcher in the field of Cross-Field in 2020. He is also named the Cybersecurity Educator of the Year-APAC (Cybersecurity Excellence Awards are produced in cooperation with the Information Security Community on LinkedIn) in 2016. In 2015, he and his team won the Digital Forensics Research Challenge organized by the Germany's University of Erlangen-Nuremberg. He was a recipient of the 2019 IEEE Technical Committee on Scalable Computing Award for Excellence in Scalable Computing (Middle Career Researcher), the 2018 UTSA College of Business Col. Jean Piccione and Lt. Col. Philip Piccione Endowed Research Award for Tenured Faculty, the Outstanding Associate Editor of 2018 for IEEE ACCESS, the British Computer Society's 2019 Wilkes Award Runner-up, the 2014 Highly Commended Award by the Australia New Zealand Policing Advisory Agency, the Fulbright Scholarship in 2009, the 2008 Australia Day Achievement Medallion, and the British Computer Society's Wilkes Award in 2008. He has also received the Best Paper Awards from the IEEE SYSTEMS JOURNAL in 2021, *IEEE Consumer Electronics Magazine* in 2020, *EURASIP Journal on Wireless Communications and Networking* in 2019, IEEE TrustCom 2018, and ESORICS 2015; the Korea Information Processing Society's *Journal of Information Processing Systems* (JIPS) Outstanding Research Award (Most-cited Paper) for 2020 and Survey Paper Award (Gold) in 2019; the IEEE Blockchain 2019 Outstanding Paper Award; and the Best Student Paper Awards from Inscrypt 2019 and ACISP 2005.

Authorized licensed use limited to: Wuhan University. Downloaded on October 13,2021 at 03:20:59 UTC from IEEE Xplore. Restrictions apply.